

Emacs

Emacs is the extensible, customizable, self-documenting real time display text editor. This reference was made for Emacs 27.

Getting started

Starting Emacs

To enter Emacs, just type its name:

```
$ emacs
```

Suspend Emacs

```
C-z
```

Exit Emacs permanently

```
C-x
```

```
C-c
```

Global Description

```
C-<key>
```

Means hold the control, and press <key>

		Motion
Backward	Forward	Entity to move over
<code>C-b</code>	<code>C-f</code>	Character
<code>M-b</code>	<code>M-f</code>	Word
<code>C-p</code>	<code>C-n</code>	Line
<code>C-a</code>	<code>C-e</code>	Line beginning (or end)
<code>M-a</code>	<code>M-e</code>	Sentence
<code>M-{</code>	<code>M-}</code>	Paragraph
<code>C-x</code> <code>[</code>	<code>C-x</code> <code>]</code>	Page
<code>C-M-b</code>	<code>C-M-f</code>	Sexpr
<code>C-M-a</code>	<code>C-M-e</code>	Function
<code>M-<</code>	<code>M-></code>	Buffer beginning (or end)

		Case Change
<code>M-u</code>		Uppercase word
<code>M-l</code>		Lowercase word
<code>M-c</code>		Capitalize word
<code>C-x</code> <code>C-u</code>		Uppercase region
<code>C-x</code> <code>C-l</code>		Lowercase region

		Files
<code>C-x</code> <code>C-f</code>		Read a file into Emacs
<code>C-x</code> <code>C-s</code>		Save a file back to disk
<code>C-x</code> <code>s</code>		Save all files
<code>C-x</code> <code>i</code>		Insert contents of another file into this buffer
<code>C-x</code> <code>C-v</code>		Replace this file with your file
<code>C-x</code> <code>C-w</code>		Write buffer to a specified file
<code>C-x</code> <code>C-q</code>		Toggle read-only status of buffer

Error Recovery

C-g

Abort partially typed or executing command

M-x recover-session

Recover files lost by a system crash

C-x u

C-_

C-/

Undo an unwanted change

M-x revert-buffer

Restore a buffer to its original contents

C-l

Redraw garbaged screen

Transposing

C-t

Transpose characters

M-t

Transpose words

C-x C-t

Transpose lines

C-M-t

Transpose sexps

Scroll

C-v

Scroll to next screen

M-v

Scroll to previous screen

C-x <

Scroll left

C-x >

Scroll right

C-l

Scroll current line to
center, top, bottom

Goto

M-g g

Goto line

M-g c

Goto char

M-m

Back to indentation

Marking

C-@ C-SPC	Uet mark here
C-x C-x	Exchange point and mark
M-@	Set mark arg words away
M-h	Mark paragraph
C-x C-p	Mark page
C-M-@	Mark sexp
C-M-h	Mark function
C-x h	Mark entire buffer

Killing and Deleting

Backward	Forward	Entity to kill
DEL	C-d	Character (delete)
M-DEL	M-d	Word
M-0 C-k	C-k	Line (to end of)
C-x DEL	M-k	Sentence
M-- C-M-k	C-M-k	Sexp
Killing		
C-W		Kill region C-w
M-w		Copy region to kill ring
M-z char		Kill through next occurrence of char
C-y		Yank back last thing killed
M-y		Replace last yank with previous kill

Getting Help

<code>C-x</code> <code>1</code>	Remove help window
<code>C-M-v</code>	Scroll help window
<code>C-h</code> <code>a</code>	Apropos: show commands matching a string
<code>C-h</code> <code>k</code>	Describe the function a key runs
<code>C-h</code> <code>f</code>	Describe a function
<code>C-h</code> <code>m</code>	Get mode-specific information

The help system is simple. Type `C-h` (or `F1`) and follow the directions. If you are a first-time user, type `C-h t` for a tutorial.

Multiple Windows

When two commands are shown, the second is a similar command for a frame instead of a window.

<code>C-x</code> <code>5</code> <code>1</code>	<code>C-x</code> <code>1</code>	Delete all other windows
<code>C-x</code> <code>5</code> <code>2</code>	<code>C-x</code> <code>2</code>	Split window, above and below
<code>C-x</code> <code>5</code> <code>0</code>	<code>C-x</code> <code>0</code>	Delete this window
	<code>C-x</code> <code>3</code>	Split window, side by side
	<code>C-M-v</code>	Scroll other window
<code>C-x</code> <code>5</code> <code>o</code>	<code>C-x</code> <code>o</code>	Switch cursor to another window
<code>C-x</code> <code>5</code> <code>b</code>	<code>C-x</code> <code>4</code> <code>b</code>	Select buffer in other window
<code>C-x</code> <code>5</code> <code>C-o</code>	<code>C-x</code> <code>4</code> <code>C-o</code>	Display buffer in other window
<code>C-x</code> <code>5</code> <code>f</code>	<code>C-x</code> <code>4</code> <code>f</code>	Find file in other window
<code>C-x</code> <code>5</code> <code>r</code>	<code>C-x</code> <code>4</code> <code>r</code>	Find file read-only in other window
<code>C-x</code> <code>5</code> <code>d</code>	<code>C-x</code> <code>4</code> <code>d</code>	Run Dired in other window
<code>C-x</code> <code>5</code> <code>.</code>	<code>C-x</code> <code>4</code> <code>.</code>	Find tag in other window
	<code>C-x</code> <code>^</code>	Grow window taller
	<code>C-x</code> <code>{</code>	Shrink window narrower
	<code>C-x</code> <code>}</code>	Grow window wider

Formatting

<code>TAB</code>	Indent current line (mode-dependent)
<code>C-M-\</code>	Indent region (mode-dependent)
<code>C-M-q</code>	Indent sexp (mode-dependent)
<code>C-x</code> <code>TAB</code>	Indent region rigidly arg columns
<code>M-;</code>	Indent for comment
<code>C-o</code>	Insert newline after point
<code>C-M-o</code>	Move rest of line vertically down
<code>C-x</code> <code>C-o</code>	Delete blank lines around point
<code>M-^</code>	Join line with previous (with arg, next)
<code>M-\</code>	Delete all white space around point
<code>M-SPC</code>	Put exactly one space at point
<code>M-q</code>	Fill paragraph
<code>C-x</code> <code>f</code>	Set fill column to arg
<code>C-x</code> <code>.</code>	Set prefix each line starts with
<code>M-o</code>	Set face

Info	
C-h i	Enter the Info documentation reader
C-h S	Find specified function or variable in Info
Moving within a node	
SPC	Scroll forward
DEL	Scroll reverse
b	Beginning of node
Moving between nodes	
n	Next node
p	Previous node
u	Move up
m	Select menu item by name
n	Select nth menu item by number (1–9)
f	Follow cross reference (return with l)
l	Return to last node you saw
d	Return to directory node
t	Go to top node of Info file
g	Go to any node by name
Other	
h	Run Info tutorial
i	Look up a subject in the indices
s	Search nodes for regexp
q	Quit Info

Minibuffer

The following keys are defined in the minibuffer.

<code>TAB</code>	Complete as much as possible
<code>SPC</code>	Complete up to one word
<code>RET</code>	Complete and execute
<code>?</code>	Show possible completions
<code>M-p</code>	Fetch previous minibuffer input
<code>M-n</code>	Fetch later minibuffer input or default
<code>M-r</code>	Regex search backward through history
<code>M-s</code>	Regex search forward through history
<code>C-g</code>	Abort command

Type `C-x ESC ESC` to edit and repeat the last command that used the minibuffer. Type `F10` to activate menu bar items on text terminals.

Tags

<code>M-.</code>	Find a tag (a definition)
<code>C-u</code> <code>M-.</code>	Find next occurrence of tag
<code>M-x</code> visit-tags-table	Specify a new tags file
<code>M-x</code> tags-search	Regex search on all files in tags table
<code>M-x</code> tags-query-replace	Run query-replace on all the files
<code>M-,</code>	Continue last tags search or query-replace

Rectangles

<code>C-x</code> <code>r</code> <code>r</code>	Copy rectangle to register
<code>C-x</code> <code>r</code> <code>k</code>	Kill rectangle
<code>C-x</code> <code>r</code> <code>y</code>	Yank rectangle
<code>C-x</code> <code>r</code> <code>o</code>	Open rectangle, shifting text right
<code>C-x</code> <code>r</code> <code>c</code>	Blank out rectangle
<code>C-x</code> <code>r</code> <code>t</code>	Prefix each line with a string

Keyboard Macros

<code>C-x</code> <code>(</code>	Start defining a keyboard macro
<code>C-x</code> <code>)</code>	End keyboard macro definition
<code>C-x</code> <code>e</code>	Execute last-defined keyboard macro
<code>C-u</code> <code>C-x</code> <code>(</code>	Append to last keyboard macro
<code>M-x</code> <code>name-last-kbd-macro</code>	Name last keyboard macro
<code>M-x</code> <code>insert-kbd-macro</code>	Insert Lisp definition in buffer

Buffers

<code>C-x</code> <code>b</code>	Select another buffer
<code>C-x</code> <code>C-b</code>	List all buffers
<code>C-x</code> <code>k</code>	Kill a buffer

Emacs Search

Regex (common)

<code>.</code> (dot)	Any single character except a newline
<code>*</code>	Zero or more repeats
<code>+</code>	One or more repeats
<code>?</code>	Zero or one repeat
<code>\</code>	Quote special characters
<code>\c</code>	Quote regular expression special character c
<code>\ </code>	Alternative ("or")
<code>\(...\)</code>	Grouping
<code>\(:?...\)</code>	Shy grouping
<code>\(:NUM... \)</code>	Explicit numbered grouping
<code>\n</code>	Same text as nth group
<code>\b</code>	At word break
<code>\B</code>	Not at word break

		Regex (entry)
Start	End	Entity
<code>^</code>	<code>\$</code>	Line
<code>\<</code>	<code>\></code>	Word
<code>_<</code>	<code>_></code>	Symbol
<code>\'</code>	<code>\'</code>	Buffer

		Regex (conflict)
These	Others	class
<code>[...]</code>	<code>[^...]</code>	Explicit set
<code>\w</code>	<code>\W</code>	Word-syntax character
<code>\sc</code>	<code>\Sc</code>	Character with syntax c
<code>\cc</code>	<code>\Cc</code>	Character with category c

		Incremental Search
<code>C-s</code>		Search forward
<code>C-r</code>		Search backward
<code>C-M-s</code>		Regular expression search
<code>C-M-r</code>		Reverse regular expression search
<code>M-p</code>		Select previous search string
<code>M-n</code>		Select next later search string
<code>RET</code>		Exit incremental search
<code>DEL</code>		Undo effect of last character
<code>C-g</code>		Abort current search

Use C-s or C-r again to repeat the search in either direction. If Emacs is still searching, C-g cancels only the part not matched.

Query Replace	
<code>M-%</code>	Interactively replace a text string
<code>M-x</code> <code>regex</code>	Using regular expressions
<code>SPC</code> / <code>y</code>	Replace this one, go on to next
<code>,</code>	Replace this one, don't move
<code>DEL</code> / <code>n</code>	Skip to next without replacing
<code>!</code>	Replace all remaining matches
<code>^</code>	Back up to the previous match
<code>RET</code>	Exit query-replace
<code>C-r</code>	Enter recursive edit (C-M-c to exit)

Misc

Shells	
<code>M-!</code>	Execute a shell command
<code>M-&</code>	Execute a shell command asynchronously
<code>M-</code>	Run a shell command on the region
<code>C-u</code> <code>M-</code>	Filter region through a shell command
<code>M-x</code> <code>shell</code>	Start a shell in window shell

International Character Sets	
<code>C-x</code> <code>RET</code> <code>I</code>	specify principal language
<code>M-x</code> <code>list-input-methods</code>	show all input methods
<code>C-\</code>	enable or disable input method
<code>C-x</code> <code>RET</code> <code>c</code>	set coding system for next command
<code>M-x</code> <code>list-coding-systems</code>	show all coding systems
<code>M-x</code> <code>prefer-coding-system</code>	choose preferred coding system

Registers

`C-x` `r` `s`

Save region in register

`C-x` `r` `i`

Insert register contents into buffer

`C-x` `r` `SPC`

Save value of point in register

`C-x` `r` `j`

Jump to point saved in register

Miscellaneous

`C-u` num

Numeric argument

`M--`

Negative argument

`C-q` char

Quoted insert

Commands Dealing with Emacs Lisp

`C-x` `C-e`

Eval sexp before point

`C-M-x`

Eval current defun

`M-x` eval-region

Eval region

`M-:`

Read and eval minibuffer

`M-x` load-library

Load a Lisp library from load-path

Simple Customization

`M-x customize`

customize variables and faces

Making global key bindings in Emacs Lisp:

```
(global-set-key (kbd "C-c g") 'search-forward)
(global-set-key (kbd "M-#") 'query-replace-regexp)
```

Abbrevs

`C-x` `a` `g`

Add global abbrev

`C-x` `a` `l`

Add mode-local abbrev

`C-x` `a` `i` `g`

Add global expansion for this abbrev

`C-x` `a` `i` `l`

Add mode-local expansion for this abbrev

`C-x` `a` `e`

Explicitly expand abbrev

`M-/`

Expand previous word dynamically

Spelling Check

M-\$

Check spelling of current word

M-x ispell-region

Check spelling of all words in region

M-x ispell-buffer

Check spelling of entire buffer

M-x flyspell-mode

Toggle on-the-fly spell checking

Writing Commands

Syntax

```
(defun command-name (args)
  "documentation" (interactive "template")
  body)
```

Example

```
(defun this-line-to-top-of-window (line)
  "Reposition current line to top of window.
With prefix argument LINE, put point on LINE."
  (interactive "P")
  (recenter (if (null line)
                0
                (prefix-numeric-value line))))
```

The interactive spec says how to read arguments interactively. Type C-h f interactive RET for more details.

Related Cheatsheet

[Vim Cheatsheet](#)[Quick Reference](#)[VSCode Cheatsheet](#)[Quick Reference](#)[Sed Cheatsheet](#)[Quick Reference](#)[Awk Cheatsheet](#)[Quick Reference](#)

Recent Cheatsheet

[InfrontJS Cheatsheet](#)[Quick Reference](#)[Remote Work Revolution Cheatsheet](#)[Quick Reference](#)[Homebrew Cheatsheet](#)[Quick Reference](#)[PyTorch Cheatsheet](#)[Quick Reference](#)

© 2026 QuickRef.ME, All rights reserved.